

Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

- **Question:** Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

Scenario 2: A customer can be promoted to VIP status based on their balance.

- **Question:** Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over \$10,000.

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

- **Question:** Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

Solution:

```
[ SQL Worksheet ]*  ▶  ⚙  🔑  📄  ⌵  Aa  🗑  📶  🔗

1  CREATE TABLE Customers (
2      CustomerID NUMBER PRIMARY KEY,
3      Name VARCHAR2(50),
4      Age NUMBER,
5      Balance NUMBER,
6      IsVIP VARCHAR2(5)
7  );
8
9  CREATE TABLE Loans (
10     LoanID NUMBER PRIMARY KEY,
11     CustomerID NUMBER,
12     InterestRate NUMBER,
13     DueDate DATE,
14     FOREIGN KEY (CustomerID)
15         REFERENCES Customers(CustomerID)
16 );
```

```
[ SQL Worksheet ]*  ▶  ⚙  🔑  📄  ⌵  Aa  🗑  📶  🔗

1  INSERT INTO Customers VALUES (1, 'Sai', 65, 15000, 'N');
2  INSERT INTO Customers VALUES (2, 'Abhi', 45, 12000, 'N');
3  INSERT INTO Customers VALUES (3, 'Ram', 70, 8000, 'N');
4
5  INSERT INTO Loans
6  VALUES (101, 1, 10, SYSDATE + 15);
7
8  INSERT INTO Loans
9  VALUES (102, 2, 12, SYSDATE + 25);
10
11 INSERT INTO Loans
12 VALUES (103, 3, 11, SYSDATE + 40);
13
14 COMMIT;
```

SELECT * FROM Customers;

SELECT * FROM Loans;

Query result Script output DBMS output Explain Plan SQL history					
  Download Execution time: 0.013 seconds					
	CUSTOMERID	NAME	AGE	BALANCE	ISVIP
1	1	Sai	65	15000	N
2	2	Abhi	45	12000	N
3	3	Ram	70	8000	N

Scenario 1:

```
BEGIN
  FOR cust_rec IN (
    SELECT CustomerID
    FROM Customers
    WHERE Age > 60
  )
  LOOP
    UPDATE Loans
    SET InterestRate = InterestRate - 1
    WHERE CustomerID = cust_rec.CustomerID;

    DBMS_OUTPUT.PUT_LINE(
      'Discount applied for Customer ID: ' ||
      cust_rec.CustomerID
    );
  END LOOP;

  COMMIT;
END;
```

```
SQL> BEGIN
      FOR cust_rec IN (
        SELECT CustomerID
        FROM Customers...
Show more...

Discount applied for Customer ID: 1
Discount applied for Customer ID: 3

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.030
```

```
SELECT LoanID, CustomerID, InterestRate
FROM Loans;
```

Query result

Script output

DBMS output

Expand

Plan

SQL history

Download

Execution time: 0.006 seconds

	LOANID	CUSTOMERID	INTERESTRATE
1	101	1	9
2	102	2	12
3	103	3	10

```
SQL> BEGIN
      FOR cust_rec IN (
        SELECT CustomerID
        FROM Customers...
```

Show more...

Customer promoted to VIP: 1

Customer promoted to VIP: 2

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.015

```
SELECT CustomerID, Name, Balance, IsVIP
FROM Customers;
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download ▼

Execution time: 0.001 seconds

	CUSTOMERID	NAME	BALANCE	ISVIP
1	1	Sai	15000	TRUE
2	2	Abhi	12000	TRUE
3	3	Ram	8000	N

Scenario 3:

```
BEGIN
  FOR loan_rec IN (
    SELECT c.Name,
           l.LoanID,
           l.DueDate
    FROM Customers c
    JOIN Loans l
      ON c.CustomerID = l.CustomerID
    WHERE l.DueDate BETWEEN SYSDATE
              AND SYSDATE + 30
  )
  LOOP
    DBMS_OUTPUT.PUT_LINE(
      'Reminder: Dear ' ||
      loan_rec.Name ||
      ', your Loan ID ' ||
      loan_rec.LoanID ||
      ' is due on ' ||
      TO_CHAR(loan_rec.DueDate, 'DD-MON-YYYY')
    );
  END LOOP;
END;
```

```
SQL> BEGIN
      FOR loan_rec IN (
        SELECT c.Name,
               l.LoanID,...
```

Show more...

```
Reminder: Dear Sai, your Loan ID 101 is due on 10-JUL-2026
Reminder: Dear Abhi, your Loan ID 102 is due on 20-JUL-2026
```

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.018

```
SELECT c.Name,
       l.LoanID,
       l.DueDate
FROM Customers c
JOIN Loans l
  ON c.CustomerID = l.CustomerID
WHERE l.DueDate BETWEEN SYSDATE AND SYSDATE + 30;
```

Query result					Script output	DBMS output	Explain Plan	SQL history
Download ▼ Execution time: 0.013 seconds								
	NAME	LOANID	DUE DATE					
1	Sai	101	7/10/2026, 11:28:5					
2	Abhi	102	7/20/2026, 11:28:5					

All outputs:

Query result					Script output	DBMS output	Explain Plan	SQL history
Discount applied for Customer ID: 1								
Discount applied for Customer ID: 3								
Customer promoted to VIP: 1								
Customer promoted to VIP: 2								
Reminder: Dear Sai, your Loan ID 101 is due on 10-JUL-2026								
Reminder: Dear Abhi, your Loan ID 102 is due on 20-JUL-2026								

Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.

- **Question:** Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

- **Question:** Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

Scenario 3: Customers should be able to transfer funds between their accounts.

- **Question:** Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

Solution:

```
CREATE TABLE Accounts (  
    AccountID NUMBER PRIMARY KEY,  
    CustomerID NUMBER,  
    AccountType VARCHAR2(20),  
    Balance NUMBER  
);
```

```
INSERT INTO Accounts VALUES (1, 101, 'SAVINGS', 10000);  
INSERT INTO Accounts VALUES (2, 102, 'SAVINGS', 5000);  
INSERT INTO Accounts VALUES (3, 103, 'CURRENT', 20000);  
COMMIT;
```

SELECT * FROM Accounts;

Query result Script output DBMS output Explain Plan SQL history					
  Download ▾ Execution time: 0.021 seconds					
	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE	
1	1	101	SAVINGS	10000	
2	2	102	SAVINGS	5000	
3	3	103	CURRENT	20000	

Scenario 1:

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
AS
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance * 0.01)
    WHERE AccountType = 'SAVINGS';

    COMMIT;
END;
/
```

```
SELECT object name, object type
FROM user_objects
WHERE object name = 'PROCESSMONTHLYINTEREST';
```

Query result Script output DBMS output Explain Plan SQL history			
  Download ▾ Execution time: 0.038 seconds			
	OBJECT_NAME	OBJECT_TYPE	
1	PROCESSMONTHLYINTEREST	PROCEDURE	

```
BEGIN
    ProcessMonthlyInterest;
END;
/
```

```
SQL> BEGIN
        ProcessMonthlyInterest;
    END;

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.005
```

```
SELECT * FROM Accounts;
```

Query result Script output DBMS output Explain Plan SQL history					
  Download ▾ Execution time: 0.008 seconds					
	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE	
1	1	101	SAVINGS	10303.01	
2	2	102	SAVINGS	5151.505	
3	3	103	CURRENT	20000	

Scenario 2:

```
CREATE TABLE Employees (  
    EmployeeID NUMBER PRIMARY KEY,  
    EmployeeName VARCHAR2(50),  
    DepartmentID NUMBER,  
    Salary NUMBER  
);
```

```
INSERT INTO Employees VALUES (1, 'Sai', 101, 50000);  
INSERT INTO Employees VALUES (2, 'Abhi', 101, 60000);  
INSERT INTO Employees VALUES (3, 'Ram', 102, 55000);  
COMMIT;
```

```
SELECT * FROM Employees;
```

Query result Script output DBMS output Explain Plan SQL history					
  Download ▾ Execution time: 0.015 seconds					
	EMPLOYEEID	EMPLOYEENAME	DEPARTMENTID	SALARY	
1	1	Sai	101	50000	
2	2	Abhi	101	60000	
3	3	Ram	102	55000	

```

CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus (
    p_DepartmentID IN NUMBER,
    p_BonusPercent IN NUMBER
)
AS
BEGIN
    UPDATE Employees
    SET Salary = Salary + (Salary * p_BonusPercent / 100)
    WHERE DepartmentID = p_DepartmentID;

    COMMIT;
END;
/

```

```

BEGIN
    UpdateEmployeeBonus(101, 10);
END;
/

```

```

SQL> BEGIN
        UpdateEmployeeBonus(101, 10);
    END;

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.017

```

```
SELECT * FROM Employees;
```

Query result Script output DBMS output Explain Plan SQL history				
  Download ▼ Execution time: 0.002 seconds				
	EMPLOYEEID	EMPLOYEEENAME	DEPARTMENTID	SALARY
1	1	Sai	101	55000
2	2	Abhi	101	66000
3	3	Ram	102	55000

Scenario 3:

```
CREATE OR REPLACE PROCEDURE TransferFunds(  
    p_FromAccount IN NUMBER,  
    p_ToAccount IN NUMBER,  
    p_Amount IN NUMBER  
)  
AS  
    v_Balance NUMBER;  
BEGIN  
  
    SELECT Balance  
    INTO v_Balance  
    FROM Accounts  
    WHERE AccountID = p_FromAccount;  
  
    IF v_Balance >= p_Amount THEN  
  
        UPDATE Accounts  
        SET Balance = Balance - p_Amount  
        WHERE AccountID = p_FromAccount;  
  
        UPDATE Accounts  
        SET Balance = Balance + p_Amount  
        WHERE AccountID = p_ToAccount;  
  
        COMMIT;  
  
        DBMS_OUTPUT.PUT_LINE('Transfer Successful');  
  
    ELSE  
  
        DBMS_OUTPUT.PUT_LINE('Insufficient Balance');  
  
    END IF;  
  
EXCEPTION  
    WHEN NO_DATA_FOUND THEN  
        DBMS_OUTPUT.PUT_LINE('Account Not Found');  
  
    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);  
END;  
/
```

```
BEGIN  
    TransferFunds(1, 2, 2000);  
END;  
/
```

Query result Script output DBMS output Explain Plan SQL history



```
SQL> BEGIN  
      TransferFunds(1, 2, 2000);  
      END;
```

Transfer Successful

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.027

```
SELECT AccountID, Balance
FROM Accounts;
```

Query result	Script output	DBMS output	Explain Plan	SQL history
  Download  Execution time: 0.002 seconds				
	EMPLOYEEID	EMPLOYEENAME	DEPARTMENTID	SALARY
1	1 Sai		101	55000
2	2 Abhi		101	66000
3	3 Ram		102	55000

```
BEGIN
    TransferFunds(1, 2, 50000);
END;
/
```

```
SQL> BEGIN
      TransferFunds(1, 2, 50000);
    END;
```

Insufficient Balance

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.010

Collapse

Script output

DBMS output

Explain Plan

SQL history

Download

Execution time: 0.002 seconds

	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE
1	1	101	SAVINGS	8303.01
2	2	102	SAVINGS	7151.505
3	3	103	CURRENT	20000